Linked List in Java — Complete Concepts

Since you want to understand it properly before moving deeper into Trees, I'll cover Linked List from zero to the important DSA level, with Java code and dry runs.

The most important idea throughout the whole topic is:

A Linked List is a collection of nodes where each node stores data and a reference to another node.

1. What is a Linked List?

Suppose we have:

```
10
20
30
40
```

In an array, these elements are stored in continuous memory conceptually:

```
[10][20][30][40]
```

A Linked List works differently.

Each element is a Node:

```
[10 | next] → [20 | next] → [30 | next] → [40 | null]
```

Each node contains two things:

```
Node
 ├── data
 └── next
```

For example:

```
[10 | address of 20]
```

The next part tells us where the next node is.

2. Why do we need Linked Lists?

Arrays have an important limitation.

Suppose:

```
[10][20][30][40][50]
```

If we want to insert 5 at the beginning:

```
[5][10][20][30][40][50]
```

In an array, existing elements may need to be shifted.

```
10 → 20 → 30 → 40 → 50
↓    ↓    ↓    ↓    ↓
20 → 30 → 40 → 50 → ...
```

That can take:

```
O(n)
```

With a Linked List, we can create a new node and change one reference:

```
Before:

10 → 20 → 30 → null

After:

5 → 10 → 20 → 30 → null
```

We don't move 10, 20, or 30.

We simply connect:

```
5 → 10
```

3. What exactly is a Node?

Let's create a Node in Java.

```java
class Node {

    int data;
    Node next;
}
```

There are two variables:

```java
int data;
```

and:

```java
Node next;
```

data

Stores the actual value.

Example:

```
10
```

next

Stores a reference to another Node.

For example:

```
Node 1
data = 10
next → Node 2
```

So:

```
[10 | next]
```

4. Creating a Node Constructor

Instead of manually assigning values, we normally use a constructor.

```java
class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Let's understand this carefully.

Node(int data)

This is the constructor.

If we write:

```java
Node node = new Node(10);
```

Java calls:

```java
Node(int data)
```

with:

```
data = 10
```

Then:

```java
this.data = data;
```

means:

```
node's data = 10
```

And:

```
this.next = null;
```

means:

```
this node currently doesn't point to another node.
```

So we get:

```
[10 | null]
```

5. What does null mean?

This is extremely important.

Suppose:

```
[10] → [20] → [30]
```

There is no node after 30.

Therefore:

```
[30 | null]
```

So:

```
10 → 20 → 30 → null
```

null means:

There is no next node.

Therefore, the last node of a normal singly linked list always points to:

```
null
```

6. Creating Multiple Nodes

We can manually create:

```
Node first = new Node(10);
Node second = new Node(20);
Node third = new Node(30);
```

Initially they are independent:

```
first

[10 | null]


second

[20 | null]


third

[30 | null]
```

Now connect them.

```
first.next = second;
second.next = third;
```

Now:

```
first
  ↓
[10] → [20] → [30] → null
```

This is a Linked List.

7. What is head?

This is probably the most important variable in a Linked List.

We normally maintain:

```
Node head;
```

head points to the first node.

Example:

```
head
 ↓
[10] → [20] → [30] → null
```

Notice something important:

head does not contain 10.

It contains a reference to the node containing 10.

Conceptually:

```
head → Node(10)
```

8. Why is head important?

Suppose:

```
head
 ↓
10 → 20 → 30 → null
```

To reach 20:

```
head
 ↓
10
 ↓
next
 ↓
20
```

To reach 30:

```
head
 ↓
10
 ↓
next
 ↓
20
 ↓
next
 ↓
30
```

The entire Linked List can be reached starting from head.

That's why:

If you lose head, you lose access to the beginning of the list.

9. Traversing a Linked List

Traversal means:

Visit every node one by one.

Suppose:

```
head
 ↓
10 → 20 → 30 → 40 → null
```

We need to visit:

```
10
20
30
40
```

We can use a temporary variable.

```java
Node current = head;
```

Initially:

```
current
   ↓
  10 → 20 → 30 → 40 → null
```

Then:

```
current = current.next;
```

Now:

```
10 → 20 → 30 → 40
      ↑
    current
```

Again:

```
current = current.next;
```

Now:

```
10 → 20 → 30 → 40
           ↑
         current
```

10. Basic Traversal Code

```
Node current = head;

while (current != null) {

    System.out.println(current.data);

    current = current.next;
}
```

Let's understand every line.

Line 1

```
Node current = head;
```

Start at the first node.

```
current
   ↓
10 → 20 → 30 → null
```

Line 2

```
while (current != null)
```

Continue while a node exists.

When:

```
current → null
```

we stop.

Line 3

```
System.out.println(current.data);
```

Print current node's data.

Line 4

```
current = current.next;
```

Move to the next node.

This line is the heart of traversal.

11. Dry Run of Traversal

List:

```
10 → 20 → 30 → null
```

Initially:

```
current = head
```

So:

```
current → 10
```

Iteration 1:

```
print 10
current = current.next
```

Now:

```
current → 20
```

Iteration 2:

```
print 20
current = current.next
```

Now:

```
current → 30
```

Iteration 3:

```
print 30
current = current.next
```

Now:

```
current → null
```

Loop stops.

Output:

```
10
20
30
```

12. Why don't we use head = head.next for traversal?

You technically could:

```
head = head.next;
```

But don't do this for normal traversal.

Why?

Suppose:

```
head
 ↓
10 → 20 → 30 → null
```

If you do:

```
head = head.next;
```

now:

```
head
 ↓
20 → 30 → null
```

You lost your reference to 10.

That's why we use:

```
Node current = head;
```

and move current.

```
head
 ↓
10 → 20 → 30

current
 ↓
10
```

Then current moves while head remains safe.

13. Linked List Class

Now let's create a proper Linked List.

```java
class LinkedList {

    Node head;

}
```

We can define Node inside it too.

```java
class LinkedList {

    class Node {

        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }

    Node head;
}
```

14. Insert at Beginning

This is one of the most important operations.

Suppose:

```
head
 ↓
10 → 20 → 30 → null
```

We want:

```
5 → 10 → 20 → 30 → null
```

Create:

```java
Node newNode = new Node(5);
```

Now:

```
newNode
   ↓
  [5]
```

We need:

```
5 → 10
```

So:

```
newNode.next = head;
```

Now:

```
newNode
   ↓
  [5] → [10] → [20] → [30] → null
          ↑
         head
```

But head still points to 10.

So finally:

```
head = newNode;
```

Now:

```
head
 ↓
[5] → [10] → [20] → [30] → null
```

15. Complete Insert-at-Beginning Code

```
void insertFirst(int value) {

    Node newNode = new Node(value);

    newNode.next = head;

    head = newNode;
}
```

Understand the order carefully.

Step 1

```
Node newNode = new Node(value);
```

Create node.

Step 2

```
newNode.next = head;
```

Connect new node to existing list.

Step 3

```
head = newNode;
```

Make new node the first node.

16. Why is the order important?

This is WRONG:

```
head = newNode;
newNode.next = head;
```

Why?

After:

```
head = newNode;
```

both point to the same node:

```
head
 ↓
5
 ↑
newNode
```

Then:

```
newNode.next = head;
```

means:

```
5 → itself
```

You lose the old list.

Correct order:

```
newNode.next = head;
head = newNode;
```

Always connect the new node before changing head.

17. Insert at End

Suppose:

```
head
 ↓
10 → 20 → 30 → null
```

We want:

```
10 → 20 → 30 → 40 → null
```

Create:

```
Node newNode = new Node(40);
```

We need to reach the last node.

The last node is:

```
30 → null
```

So traverse:

```
Node current = head;

while (current.next != null) {
    current = current.next;
}
```

Notice:

```
current.next != null
```

not:

```
current != null
```

Why?

Because we want current to stop at the last node.

At the last node:

```
current
   ↓
  30 → null
       ↑
    current.next
```

Here:

```
current.next == null
```

So the loop stops.

Then:

```
current.next = newNode;
```

Result:

```
10 → 20 → 30 → 40 → null
```

18. Complete Insert-at-End

```java
void insertLast(int value) {

    Node newNode = new Node(value);

    if (head == null) {
        head = newNode;
        return;
    }

    Node current = head;

    while (current.next != null) {
        current = current.next;
    }

    current.next = newNode;
}
```

19. Why do we check head == null?

Suppose the list is empty:

```
head → null
```

We cannot do:

```java
current = head;
```

and then access:

```java
current.next
```

because current is null.

So:

```java
if (head == null) {
    head = newNode;
    return;
}
```

handles the empty list.

After insertion:

```
head
 ↓
10 → null
```

20. Insert at a Specific Position

Suppose:

```
10 → 20 → 30 → 40
```

We want to insert 25 after 20.

Result:

```
10 → 20 → 25 → 30 → 40
```

We need to stop at:

```
20
```

Then:

```
20 → 30
```

must become:

```
20 → 25 → 30
```

Create:

```
Node newNode = new Node(25);
```

Then:

```
newNode.next = current.next;
```

This makes:

```
25 → 30
```

Then:

```
current.next = newNode;
```

Now:

```
20 → 25 → 30
```

21. Insert at Position Code

Assuming position starts from 0:

```
void insertAt(int value, int position) {

    if (position == 0) {
        insertFirst(value);
        return;
    }

    Node newNode = new Node(value);

    Node current = head;

    for (int i = 0; i < position - 1 && current != null; i++) {
        current = current.next;
    }

    if (current == null) {
        return;
    }

    newNode.next = current.next;
    current.next = newNode;
}
```

The important part is:

```
newNode.next = current.next;
current.next = newNode;
```

This pattern appears repeatedly in Linked List problems.

22. Deleting the First Node

Suppose:

```
head
 ↓
10 → 20 → 30 → null
```

We want to remove 10.

Simply:

```
head = head.next;
```

Before:

```
head → 10 → 20 → 30
```

After:

```
head → 20 → 30
```

The old 10 node is no longer reachable from the list.

Java's garbage collector can eventually clean it up.

23. Delete First Code

```
void deleteFirst() {

    if (head == null) {
        return;
    }

    head = head.next;
}
```

Time:

```
O(1)
```

Because we don't traverse the list.

24. Delete Last Node

Suppose:

```
10 → 20 → 30 → 40 → null
```

We want:

```
10 → 20 → 30 → null
```

We need to reach the second-last node:

```
30 → 40
```

Why?

Because we need:

```
current.next = null;
```

So:

```
Node current = head;

while (current.next.next != null) {
    current = current.next;
}
```

At the end:

```
current
   ↓
30 → 40 → null
```

Then:

```
current.next = null;
```

Result:

```
10 → 20 → 30 → null
```

25. Important Edge Case: One Node

Suppose:

```
head
 ↓
10 → null
```

If we use:

```
current.next.next
```

then:

```
current.next
```

is already null.

We cannot access:

```
null.next
```

So we must handle:

```
if (head.next == null) {
    head = null;
    return;
}
```

26. Complete Delete-Last

```java
void deleteLast() {

    if (head == null) {
        return;
    }

    if (head.next == null) {
        head = null;
        return;
    }

    Node current = head;

    while (current.next.next != null) {
        current = current.next;
    }

    current.next = null;
}
```

27. Delete a Specific Value

Suppose:

```
10 → 20 → 30 → 40
```

Delete:

```
30
```

We need:

```
20 → 30 → 40
```

to become:

```
20 → 40
```

So we need the node before the node we want to delete.

```
10 → 20 → 30 → 40
     ↑    ↑
   prev  delete
```

Then:

```java
prev.next = deleteNode.next;
```

Result:

```
10 → 20 → 40
```

28. Delete Value Code

```
void deleteValue(int value) {

    if (head == null) {
        return;
    }

    if (head.data == value) {
        head = head.next;
        return;
    }

    Node current = head;

    while (current.next != null &&
           current.next.data != value) {

        current = current.next;
    }

    if (current.next != null) {
        current.next = current.next.next;
    }
}
```

The key line:

```
current.next = current.next.next;
```

Suppose:

```
current
   ↓
20 → 30 → 40
```

Then:

```
current.next
```

is 30.

And:

```
current.next.next
```

is 40.

So:

```
current.next = current.next.next;
```

becomes:

```
20 → 40
```

The 30 node is disconnected.

29. Searching in a Linked List

Suppose:

```
10 → 20 → 30 → 40
```

Search:

```
30
```

We traverse.

```java
boolean search(int value) {

    Node current = head;

    while (current != null) {

        if (current.data == value) {
            return true;
        }

        current = current.next;
    }

    return false;
}
```

Dry run:

```
current → 10
```

Is:

```
10 == 30?
```

No.

Move:

```
current → 20
```

No.

Move:

```
current → 30
```

Yes.

Return:

```
true
```

Time:

```
O(n)
```

30. Finding the Length

```java
int length() {

    int count = 0;

    Node current = head;

    while (current != null) {

        count++;

        current = current.next;
    }

    return count;
}
```

For:

```
10 → 20 → 30 → null
```

count becomes:

```
0
1
2
3
```

Return:

```
3
```

31. Complete Basic Singly Linked List

Putting the fundamental operations together:

```java
class MyLinkedList {

    class Node {

        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }

    Node head;

    void insertFirst(int value) {
```

```
        Node newNode = new Node(value);

        newNode.next = head;

        head = newNode;
    }

    void insertLast(int value) {

        Node newNode = new Node(value);

        if (head == null) {
            head = newNode;
            return;
        }

        Node current = head;

        while (current.next != null) {
            current = current.next;
        }

        current.next = newNode;
    }

    void deleteFirst() {

        if (head == null) {
            return;
        }

        head = head.next;
    }

    void deleteLast() {

        if (head == null) {
            return;
        }

        if (head.next == null) {
            head = null;
            return;
        }

        Node current = head;

        while (current.next.next != null) {
            current = current.next;
        }

        current.next = null;
    }

    boolean search(int value) {

        Node current = head;

        while (current != null) {

            if (current.data == value) {
                return true;
```

```
            }

            current = current.next;
        }

        return false;
    }

    int length() {

        int count = 0;

        Node current = head;

        while (current != null) {

            count++;

            current = current.next;
        }

        return count;
    }

    void display() {

        Node current = head;

        while (current != null) {

            System.out.print(current.data + " ");

            current = current.next;
        }

        System.out.println();
    }
}
```

32. Time Complexity of Singly Linked List

Operation	Complexity
Insert at beginning	O(1)
Delete at beginning	O(1)
Search	O(n)
Access by index	O(n)
Insert at end	O(n)
Delete at end	O(n)
Traverse	O(n)

If we maintain a tail pointer, insertion at the end can be made O(1), although deleting the last node in a singly linked list still requires finding its predecessor.

33. Why Can't We Access list[5]?

Arrays support:

```
arr[5]
```

because arrays provide direct indexing.

Linked Lists don't.

Suppose:

```
head
 ↓
10 → 20 → 30 → 40 → 50
```

To reach 50, we must travel:

```
10
 ↓
20
 ↓
30
 ↓
40
 ↓
50
```

So accessing an element by index takes:

```
O(n)
```

There is no direct:

```
list[4]
```

for a custom Linked List.

34. Singly Linked List

The Linked List we've been studying is a:

Singly Linked List

Each node has:

```
data
next
```

Diagram:

```
10 → 20 → 30 → 40 → null
```

You can move only forward.

```
10 → 20 → 30 → 40
```

You cannot directly go:

```
40 → 30
```

because 40 doesn't store a reference to 30.

35. Doubly Linked List

A Doubly Linked List has:

```
prev
data
next
```

Example:

```
null ← 10 ⇄ 20 ⇄ 30 → null
```

Each node knows:

```
previous node
next node
```

Node:

```
class Node {

    int data;

    Node prev;
    Node next;

    Node(int data) {
        this.data = data;
    }
}
```

36. Why use Doubly Linked List?

Suppose:

```
10 ⇄ 20 ⇄ 30
```

From 20, we can move:

```
20 → 30
```

using:

```
next
```

or:

```
20 → 10
```

using:

```
prev
```

So movement is possible in both directions.

37. Doubly Linked List Insertion

Suppose:

```
10 ⇄ 20 ⇄ 30
```

Insert 25 between 20 and 30.

Initially:

```
20 ⇄ 30
```

We want:

```
20 ⇄ 25 ⇄ 30
```

We need several links.

```
newNode.prev = current;
newNode.next = current.next;

current.next.prev = newNode;
current.next = newNode;
```

The important thing is that both directions must remain correct.

38. Circular Linked List

In a normal Linked List:

```
10 → 20 → 30 → null
```

In a Circular Linked List:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

The last node points back to the first node.

So:

```
last.next = head;
```

instead of:

```
last.next = null;
```

39. Why Circular Linked Lists?

Circular Linked Lists are useful when we repeatedly cycle through elements.

Examples include:

- Round-robin scheduling
- Circular buffers/concepts
- Repeated cyclic processing

There is no natural null at the end.

Therefore traversal must be handled carefully.

You cannot simply use:

```
while (current != null)
```

because current will never become null.

40. Traversing a Circular Linked List

Suppose:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

We can use:

```
Node current = head;

do {

    System.out.println(current.data);

    current = current.next;

} while (current != head);
```

Why do-while?

Because we need to process the first node before checking whether we've returned to head.

41. Linked List Reversal

Now we reach one of the most important Linked List algorithms for LeetCode.

Suppose:

```
1 → 2 → 3 → 4 → null
```

We want:

```
4 → 3 → 2 → 1 → null
```

This is called:

Reverse Linked List.

42. The Three Important Variables

The standard iterative solution uses:

```java
Node prev = null;
Node current = head;
Node next;
```

Initially:

```
prev
 ↓
null

current
 ↓
1 → 2 → 3 → 4 → null
```

43. Why do we need next?

Suppose:

```
current
   ↓
1 → 2 → 3 → 4
```

We want to reverse:

```
1 → 2
```

into:

```
1 ← 2
```

But if we immediately do:

```java
current.next = prev;
```

then:

```
1 → null
```

We lose access to:

```
2 → 3 → 4
```

Therefore, save the next node first.

```
next = current.next;
```

Now:

```
next
 ↓
2 → 3 → 4
```

44. Reverse Step 1

Initial:

```
prev = null

current
 ↓
1 → 2 → 3 → 4 → null
```

Save next:

```
next = current.next;
```

Now:

```
next
 ↓
2 → 3 → 4
```

Reverse pointer:

```
current.next = prev;
```

Now:

```
1 → null
```

Move prev:

```
prev = current;
```

Now:

```
prev
 ↓
1 → null
```

Move current:

```
current = next;
```

Now:

```
prev       current
 ↓           ↓
1 → null    2 → 3 → 4
```

45. Reverse Step 2

Save:

```
next = current.next;
```

Now:

```
next → 3
```

Reverse:

```
current.next = prev;
```

Now:

```
2 → 1 → null
```

Move:

```
prev = current;
current = next;
```

Now:

```
prev
 ↓
2 → 1 → null

current
 ↓
3 → 4 → null
```

Repeat.

46. Complete Reverse Code

```
Node reverse(Node head) {

    Node prev = null;
    Node current = head;

    while (current != null) {

        Node next = current.next;

        current.next = prev;

        prev = current;

        current = next;
    }

    return prev;
}
```

The four core lines are:

```
Node next = current.next;

current.next = prev;

prev = current;

current = next;
```

Memorize the logic, not just the code:

```
SAVE
REVERSE
MOVE PREV
MOVE CURRENT
```

47. Finding the Middle Node

Another extremely important pattern.

Suppose:

```
1 → 2 → 3 → 4 → 5
```

Middle:

```
3
```

Instead of counting length first, we can use:

Slow and Fast Pointers.

48. Slow and Fast Pointers

Create:

```
Node slow = head;
Node fast = head;
```

Then:

```
while (fast != null && fast.next != null) {

    slow = slow.next;

    fast = fast.next.next;
}
```

slow moves:

```
1 step
```

fast moves:

```
2 steps
```

49. Middle Dry Run

List:

```
1 → 2 → 3 → 4 → 5
```

Initially:

```
slow = 1
fast = 1
```

Iteration:

```
slow = 2
fast = 3
```

Next:

```
slow = 3
fast = 5
```

Next loop check:

```
fast.next == null
```

Stop.

Therefore:

```
slow = 3
```

Middle found.

50. Why Does This Work?

Imagine two runners.

Runner A:

```
slow → 1 step
```

Runner B:

```
fast → 2 steps
```

When fast reaches the end, slow has travelled approximately half the distance.

Therefore:

```
fast → end
slow → middle
```

This pattern is used everywhere in Linked List problems.

51. Detecting a Cycle

Consider:

```
1 → 2 → 3 → 4
        ↑     ↓
        └─────┘
```

There is no null.

The list loops forever.

This is called a:

Cycle.

A famous algorithm for detecting it is:

Floyd's Cycle Detection Algorithm

also called:

Tortoise and Hare Algorithm

52. Cycle Detection

Use:

```java
Node slow = head;
Node fast = head;
```

Then:

```
while (fast != null && fast.next != null) {

    slow = slow.next;

    fast = fast.next.next;

    if (slow == fast) {
        return true;
    }
}

return false;
```

53. Why Does Cycle Detection Work?

Without a cycle:

```
1 → 2 → 3 → 4 → null
```

Fast eventually reaches:

```
null
```

So:

```
fast == null
```

and we return:

```
false
```

With a cycle:

```
1 → 2 → 3 → 4
        ↑     ↓
        └─────┘
```

Fast and slow keep moving around the cycle.

Eventually:

```
slow == fast
```

Therefore:

```
cycle exists
```

54. Find the Starting Point of a Cycle

This is a more advanced but very important problem.

Example:

```
1 → 2 → 3 → 4 → 5
        ↑         ↓
        └─────────┘
```

Cycle begins at:

```
3
```

After detecting:

```
slow == fast
```

reset:

```
slow = head;
```

Then move both one step:

```
while (slow != fast) {

    slow = slow.next;
    fast = fast.next;
}
```

When they meet again:

```
slow == fast
```

That node is the cycle beginning.

55. Remove a Node Without Head Reference

This is a famous LeetCode-style trick.

Suppose:

```
10 → 20 → 30 → 40
```

We want to delete:

```
30
```

But we are given only the node:

```
30
```

not head.

Normally we'd need the previous node.

But we can copy the next node's data.

Current:

```
30 → 40
```

Do:

```
node.data = node.next.data;
node.next = node.next.next;
```

So:

```
30
```

becomes:

```
40
```

and the original 40 node is skipped.

Result:

```
10 → 20 → 40
```

This is the idea behind LeetCode 237 — Delete Node in a Linked List.

56. Merge Two Sorted Linked Lists

Another major problem.

Suppose:

```
List 1:

1 → 3 → 5

List 2:

2 → 4 → 6
```

We want:

```
1 → 2 → 3 → 4 → 5 → 6
```

Use two pointers:

```
p1 → 1
p2 → 2
```

Compare:

```
1 < 2
```

take 1.

Then:

```
p1 → 3
p2 → 2
```

Compare:

```
3 > 2
```

take 2.

Continue.

57. Dummy Node Technique

A very useful Linked List technique is:

Dummy Node

Instead of worrying about the first node separately, create:

```java
Node dummy = new Node(0);
Node current = dummy;
```

Then build the answer after it.

Example:

```
dummy
 ↓
0 → 1 → 2 → 3 → ...
```

At the end:

```java
return dummy.next;
```

Why?

Because dummy is just a temporary starting point.

58. Merge Two Sorted Lists Code

```
Node merge(Node list1, Node list2) {

    Node dummy = new Node(0);

    Node current = dummy;

    while (list1 != null && list2 != null) {

        if (list1.data <= list2.data) {

            current.next = list1;

            list1 = list1.next;

        } else {

            current.next = list2;

            list2 = list2.next;
        }

        current = current.next;
    }

    if (list1 != null) {
        current.next = list1;
    } else {
        current.next = list2;
    }

    return dummy.next;
}
```

This pattern is extremely important.

59. Palindrome Linked List

Suppose:

```
1 → 2 → 2 → 1
```

This is a palindrome.

We need to determine whether:

```
forward == reverse
```

A common O(n) / O(1)-extra-space approach is:

- Find middle.
- Reverse second half.
- Compare both halves.
- Optionally restore the list.

Conceptually:

```
1 → 2 | 2 → 1
```

Reverse second half:

```
1 → 2 | 1 → 2
```

Compare:

```
1 == 1
2 == 2
```

Therefore palindrome.

60. Remove Nth Node From End

Suppose:

```
1 → 2 → 3 → 4 → 5
```

Remove the 2nd node from the end:

```
1 → 2 → 3 → 5
```

The answer is commonly solved using:

Two pointers + dummy node.

Move fast ahead by n nodes.

Then move both:

```
slow
fast
```

until fast reaches the end.

Now slow is positioned just before the node that needs to be deleted.

Then:

```
slow.next = slow.next.next;
```

This is another extremely important Linked List pattern.

61. Important Linked List Patterns

You should recognize these patterns immediately.

Pattern 1 — Traversal

```
Node current = head;

while (current != null) {

    current = current.next;
}
```

Pattern 2 — Reverse

```
Node prev = null;
Node current = head;

while (current != null) {

    Node next = current.next;

    current.next = prev;

    prev = current;

    current = next;
}
```

Pattern 3 — Slow/Fast

```
Node slow = head;
Node fast = head;

while (fast != null && fast.next != null) {

    slow = slow.next;
    fast = fast.next.next;
}
```

Used for:

- Middle node
- Cycle detection
- Palindrome
- Other two-pointer problems

Pattern 4 — Previous Node

When deleting:

```
10 → 20 → 30
```

we often need:

```
previous → node-to-delete
```

Then:

```
previous.next = previous.next.next;
```

Pattern 5 — Dummy Node

```
Node dummy = new Node(0);
Node current = dummy;
```

At the end:

```
return dummy.next;
```

Very useful for:

- Merge lists
- Removing nodes
- Building a result list

62. Java's Built-in LinkedList

Java already provides:

```
LinkedList<Integer> list = new LinkedList<>();
```

Import:

```
import java.util.LinkedList;
```

Example:

```
LinkedList<Integer> list = new LinkedList<>();

list.add(10);
list.add(20);
list.add(30);
```

Now:

```
[10, 20, 30]
```

63. Important Java LinkedList Methods

Add at end

```
list.add(10);
```

Add at beginning

```
list.addFirst(10);
```

Add at end

```
list.addLast(20);
```

Remove first

```
list.removeFirst();
```

Remove last

```
list.removeLast();
```

Get first

```
list.getFirst();
```

Get last

```
list.getLast();
```

Size

```
list.size();
```

Contains

```
list.contains(20);
```

64. LinkedList vs ArrayList

This is also important.

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Random access	O(1)	O(n)
Add at end	Amortized O(1)	O(1)
Add/remove at beginning	O(n)	O(1)
Get by index	O(1)	O(n)
Memory overhead	Lower	Higher

Important distinction:

Java's LinkedList is a doubly linked list.

It is not the same as the custom singly linked list we implemented above.

65. Singly vs Doubly vs Circular

Singly

```
10 → 20 → 30 → null
```

Each node:

```
data + next
```

Doubly

```
null ← 10 ⇄ 20 ⇄ 30 → null
```

Each node:

```
prev + data + next
```

Circular Singly

```
10 → 20 → 30
↑          ↓
└──────────┘
```

Last node points to first.

Circular Doubly

```
      ┌──────────────────────┐
      ↓                      │
10 ⇄ 20 ⇄ 30 ⇄ 40
↑                        ↓
└────────────────────────┘
```

Both directions are connected circularly.

66. Linked List vs Array

Feature	Array	Linked List
Memory	Contiguous	Nodes can be scattered
Random access	O(1)	O(n)
Insert beginning	O(n)	O(1)
Delete beginning	O(n)	O(1)
Search	O(n)	O(n)
Extra pointer memory	No	Yes
Size	Fixed/basic array	Dynamic

The important trade-off is:

```
Array
↓
Fast indexing

Linked List
↓
Flexible insertion/deletion
```

67. The Most Important Pointer Understanding

If you understand this:

```
current = current.next;
```

you understand the basic movement through a Linked List.

If you understand:

```
newNode.next = head;
head = newNode;
```

you understand insertion at the beginning.

If you understand:

```
current.next = current.next.next;
```

you understand a fundamental deletion pattern.

If you understand:

```
Node next = current.next;
current.next = prev;
prev = current;
current = next;
```

you understand Linked List reversal.

These are the core pointer manipulations.

68. Common Mistakes

Mistake 1

Forgetting:

```
current = current.next;
```

This causes an infinite loop.

Mistake 2

Changing head during traversal.

Avoid:

```
head = head.next;
```

Use:

```
Node current = head;
```

Mistake 3

Not handling an empty list.

Always consider:

```
head == null
```

Mistake 4

Not handling a one-node list.

Consider:

```
10 → null
```

especially when deleting the last node.

Mistake 5

Losing the next node during reversal.

Wrong:

```
current.next = prev;
current = current.next;
```

You just changed the pointer and may lose the rest of the list.

Correct:

```
Node next = current.next;

current.next = prev;

current = next;
```

69. Complexity You Should Remember

For a basic singly linked list:

Operation	Time
Insert first	O(1)
Delete first	O(1)
Insert last	O(n)
Delete last	O(n)
Search	O(n)
Access index	O(n)
Traverse	O(n)
Reverse	O(n)
Find middle	O(n)
Detect cycle	O(n)

Extra space for these pointer algorithms such as reversal, middle finding, and cycle detection is generally:

```
O(1)
```

apart from the nodes already present in the list.

70. Important LeetCode Linked List Problems

Once your implementation basics are comfortable, these are the important patterns to practice.

Basic

206 — Reverse Linked List

Learn:

```
prev/current/next
```

Middle

876 — Middle of the Linked List

Learn:

```
slow/fast
```

Cycle

141 — Linked List Cycle

Learn:

```
Floyd's algorithm
```

Cycle Beginning

142 — Linked List Cycle II

Learn:

```
slow/fast + reset
```

Merge

21 — Merge Two Sorted Lists

Learn:

```
dummy node
```

Remove

19 — Remove Nth Node From End of List

Learn:

```
two pointers + dummy
```

Palindrome

234 — Palindrome Linked List

Learn:

```
middle
+
reverse
+
compare
```

Intersection

160 — Intersection of Two Linked Lists

Learn:

```
two-pointer switching
```

Delete Node

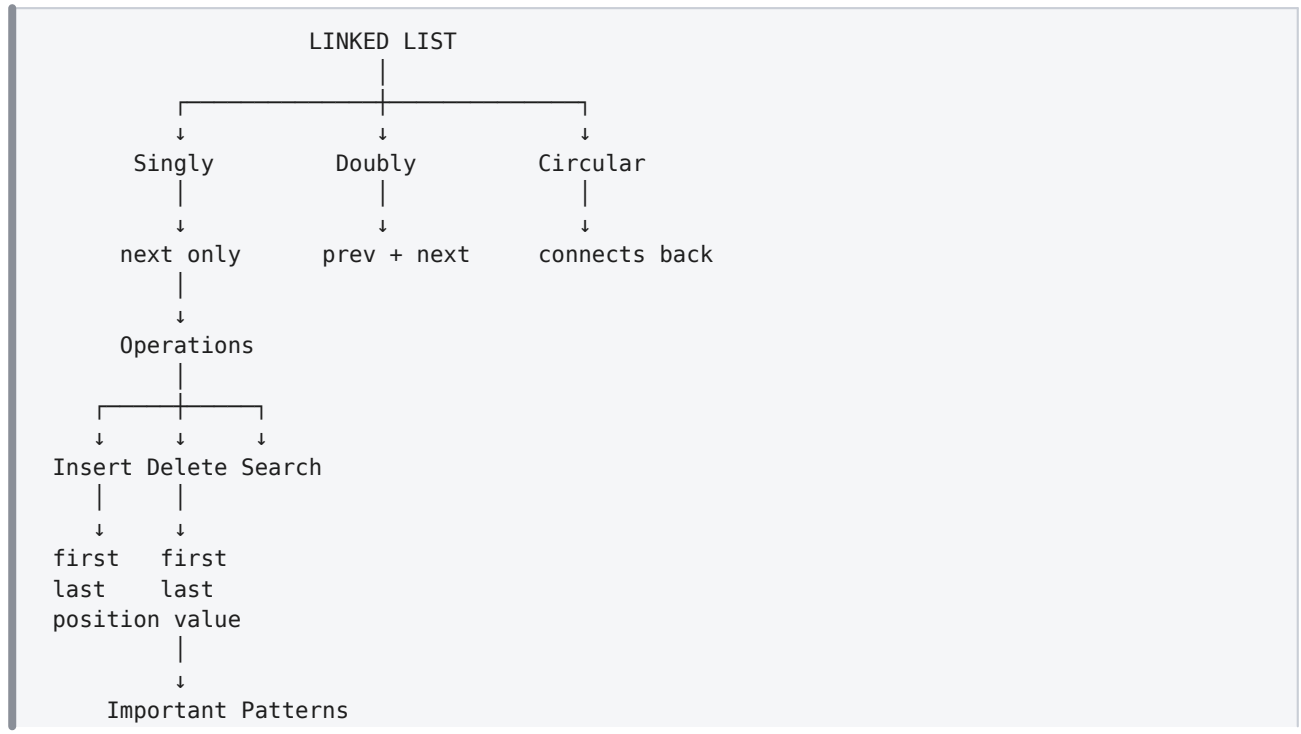
237 — Delete Node in a Linked List

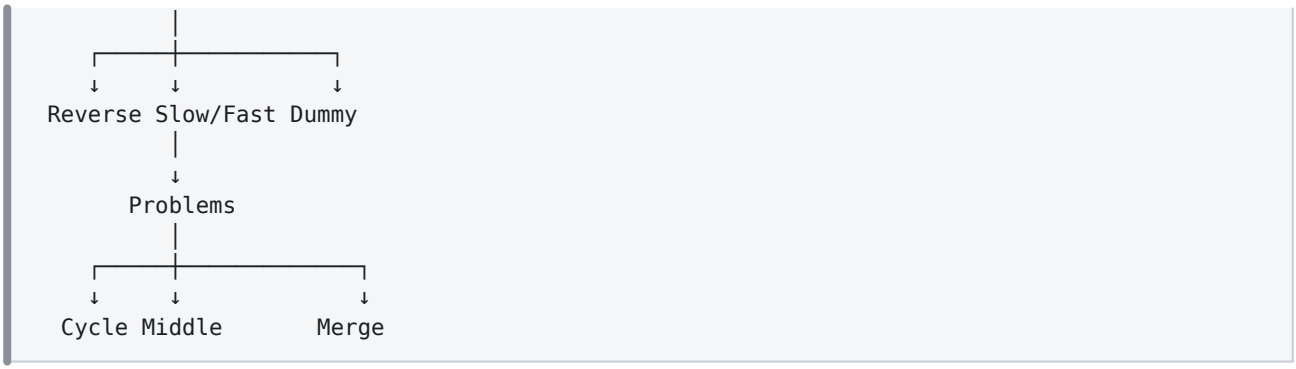
Learn:

```
copy next node
```

71. The Big Picture

You should mentally organize Linked List like this:



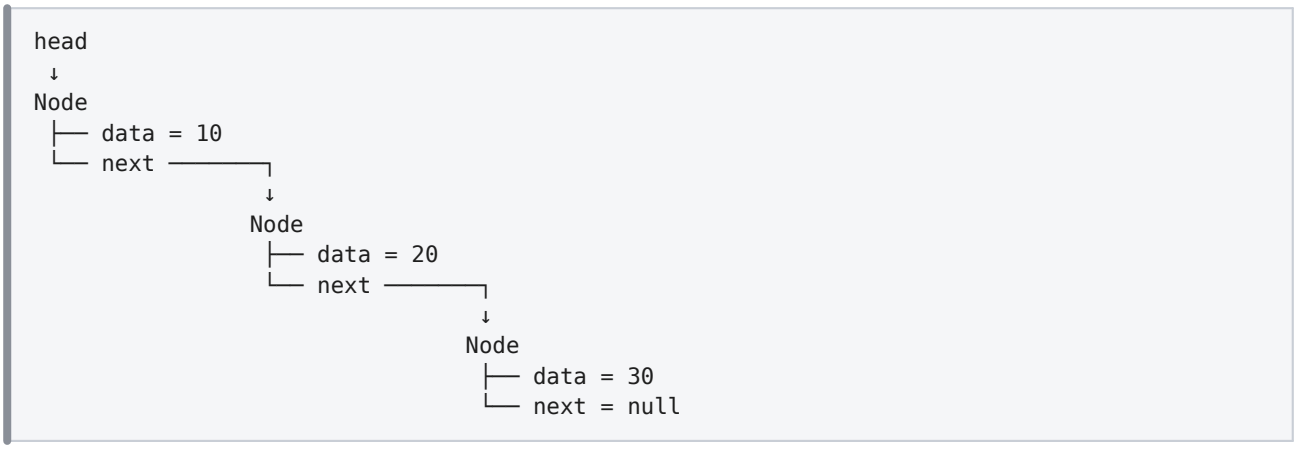


72. The Linked List Mental Model

Don't think of a Linked List as:

```
[10][20][30][40]
```

Think of it as objects pointing to other objects:



That is the actual mental model you need.

73. The 10 Things You Must Be Able to Code Yourself

Before moving forward to Trees, you should be comfortable writing these without copying.

1. Create a Node

```java
class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

2. Traverse

```
Node current = head;

while (current != null) {
    System.out.println(current.data);
    current = current.next;
}
```

3. Insert at beginning

```
newNode.next = head;
head = newNode;
```

4. Insert at end

```
while (current.next != null) {
    current = current.next;
}

current.next = newNode;
```

5. Delete first

```
head = head.next;
```

6. Delete a node

```
current.next = current.next.next;
```

7. Search

```
while (current != null) {

    if (current.data == value) {
        return true;
    }

    current = current.next;
}
```

8. Reverse

```
Node next = current.next;

current.next = prev;

prev = current;

current = next;
```

9. Find middle

```
slow = slow.next;
fast = fast.next.next;
```

10. Detect cycle

```
if (slow == fast) {
    return true;
}
```

If these ten ideas are clear, most Linked List problems stop looking like completely new problems. They become combinations of these patterns.

74. Final Cheat Sheet

```
LINKED LIST
│
├── Node
│    ├── data
│    └── next
│
├── Head
│    └── points to first node
│
├── Singly Linked List
│    └── next
│
├── Doubly Linked List
│    ├── prev
│    └── next
│
├── Circular Linked List
│    └── last → first
│
├── Basic Operations
│    ├── Insert
│    ├── Delete
│    ├── Search
│    └── Traverse
│
├── Important Patterns
│    ├── prev/current/next
│    ├── slow/fast
│    ├── dummy node
│    └── previous node
│
└── Major Problems
     ├── Reverse
     ├── Middle
     ├── Cycle
     ├── Merge
     ├── Palindrome
     ├── Remove Nth
     └── Intersection
```

The single most important thing

When you see:

```
10 → 20 → 30 → 40 → null
```

always ask yourself:

"Which node does this variable currently point to, and what does its next point to?"

Once you can mentally track those two things, Linked List becomes much easier.

Doubly and Circular Linked Lists — Detailed Coverage

Yes — you're right. In the Big Picture, I had only thoroughly covered the Singly Linked List. The Doubly and Circular sections were only an introduction.

So let's continue properly from there. We'll do Doubly Linked List first, then Circular Linked List, with the same level of detail as Singly Linked List.

Part 1 — Doubly Linked List

1. What is a Doubly Linked List?

In a Singly Linked List, every node contains:

```
data
next
```

Example:

```
10 → 20 → 30 → null
```

The node knows only about the next node.

A Doubly Linked List gives every node two links:

```
prev
data
next
```

So:

```
null ← 10 ⇄ 20 ⇄ 30 → null
```

Each node can move:

```
forward  →
backward ←
```

That's the main difference.

2. Singly vs Doubly

Singly

```
[10 | next] → [20 | next] → [30 | null]
```

From 20, you can do:

```
20 → 30
```

but you cannot directly do:

```
20 → 10
```

because 20 doesn't know where 10 is.

Doubly

```
null ← [10] ⇄ [20] ⇄ [30] → null
```

The 20 node contains:

```
prev → 10
data → 20
next → 30
```

So it can move in both directions.

3. Structure of a Doubly Node

Let's create it in Java.

```java
class Node {

    int data;

    Node prev;
    Node next;

    Node(int data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

There are three important pieces.

data

```java
int data;
```

Stores the value.

prev

```
Node prev;
```

Points to the previous node.

next

```
Node next;
```

Points to the next node.

4. Understanding One Node

Suppose:

```
Node node = new Node(20);
```

Initially:

```
        ┌──────────────┐
        │    Node      │
        ├──────────────┤
prev →  │ null         │
        │ data = 20    │
next →  │ null         │
        └──────────────┘
```

So initially:

```
prev = null
data = 20
next = null
```

5. Connecting Three Nodes

Create:

```
Node first = new Node(10);
Node second = new Node(20);
Node third = new Node(30);
```

Initially:

```
10 → null

20 → null

30 → null
```

Now connect the first and second.

```
first.next = second;
second.prev = first;
```

Now:

```
null ← 10 ⇄ 20 → null
```

Notice that we needed two assignments.

```
first.next = second;
```

means:

```
10 → 20
```

and:

```
second.prev = first;
```

means:

```
20 → 10
```

Together:

```
10 ⇄ 20
```

6. Connecting the Third Node

```
second.next = third;
third.prev = second;
```

Now:

```
null ← 10 ⇄ 20 ⇄ 30 → null
```

This is a Doubly Linked List.

7. What is head?

Same idea as Singly Linked List.

```
Node head;
```

head points to the first node.

```
head
 ↓
10 ⇄ 20 ⇄ 30
```

The first node's prev must be:

```
null
```

So:

```
head
 ↓
null ← 10 ⇄ 20 ⇄ 30 → null
```

8. What is tail?

With Doubly Linked Lists, it is useful to maintain both:

```
Node head;
Node tail;
```

head:

```
points to first node
```

tail:

```
points to last node
```

Example:

```
head                    tail
 ↓                       ↓
10 ⇄ 20 ⇄ 30 ⇄ 40 ⇄ 50
```

This is very useful because we can reach the last node directly.

9. Why is tail useful?

In our basic Singly Linked List, inserting at the end required:

```
head
 ↓
10 → 20 → 30 → 40
              ↑
          traverse
```

We had to traverse the entire list.

But with:

```
Node tail;
```

we already know where the last node is.

```
head                tail
 ↓                   ↓
10 ⇄ 20 ⇄ 30 ⇄ 40
```

So insertion at the end can be:

```
O(1)
```

10. Insert at Beginning

Suppose:

```
head
 ↓
10 ⇄ 20 ⇄ 30
```

We want:

```
5 ⇄ 10 ⇄ 20 ⇄ 30
```

Create:

```
Node newNode = new Node(5);
```

Initially:

```
5
```

We need:

```
5 ⇄ 10
```

So:

```
newNode.next = head;
```

Now:

```
5 → 10 ⇄ 20 ⇄ 30
```

But 10 doesn't point backward to 5 yet.

So:

```
head.prev = newNode;
```

Now:

```
5 ⇄ 10 ⇄ 20 ⇄ 30
```

Finally:

```
head = newNode;
```

11. Complete Insert-First

```
void insertFirst(int value) {

    Node newNode = new Node(value);

    if (head == null) {
        head = tail = newNode;
        return;
    }

    newNode.next = head;

    head.prev = newNode;

    head = newNode;
}
```

Let's understand the empty-list case.

If:

```
head = null
tail = null
```

and we insert 10:

```
head
 ↓
10
 ↑
tail
```

So:

```
head = tail = newNode;
```

Both point to the same node.

12. Why Do We Need head.prev?

This is one of the biggest differences from Singly Linked List.

Suppose:

```
10 ⇄ 20 ⇄ 30
```

We insert 5.

If we only do:

```
newNode.next = head;
head = newNode;
```

we get:

```
5 → 10 ⇄ 20 ⇄ 30
```

But:

```
10.prev
```

is still null.

That's incorrect.

We need:

```
head.prev = newNode;
```

so:

```
5 ⇄ 10 ⇄ 20 ⇄ 30
```

Whenever you modify a Doubly Linked List, think:

Do both directions still point correctly?

13. Insert at End

Suppose:

```
head             tail
 ↓                 ↓
10 ⇄ 20 ⇄ 30
```

We want:

```
10 ⇄ 20 ⇄ 30 ⇄ 40
```

Create:

```
Node newNode = new Node(40);
```

Because we have tail, we don't need traversal.

First:

```
tail.next = newNode;
```

Now:

```
10 ⇄ 20 ⇄ 30 → 40
```

But 40 doesn't point backward.

So:

```
newNode.prev = tail;
```

Now:

```
10 ⇄ 20 ⇄ 30 ⇄ 40
```

Finally:

```
tail = newNode;
```

14. Complete Insert-Last

```
void insertLast(int value) {

    Node newNode = new Node(value);

    if (head == null) {
        head = tail = newNode;
        return;
    }

    tail.next = newNode;

    newNode.prev = tail;

    tail = newNode;
}
```

Notice:

```
tail.next = newNode;
newNode.prev = tail;
tail = newNode;
```

These three lines are the important part.

15. Traversing Forward

Exactly like Singly Linked List:

```
Node current = head;

while (current != null) {

    System.out.println(current.data);

    current = current.next;
}
```

For:

```
10 ⇄ 20 ⇄ 30 ⇄ 40
```

output:

```
10
20
30
40
```

16. Traversing Backward

This is where Doubly Linked List becomes interesting.

Because we have prev, we can start at tail.

```
Node current = tail;

while (current != null) {

    System.out.println(current.data);

    current = current.prev;
}
```

For:

```
head                 tail
 ↓                     ↓
10 ⇄ 20 ⇄ 30 ⇄ 40
```

Start:

```
current → 40
```

Then:

```
current → 30
```

Then:

```
current → 20
```

Then:

```
current → 10
```

Then:

```
current → null
```

Output:

```
40
30
20
10
```

A Singly Linked List cannot do this directly.

17. Delete First

Suppose:

```
head
 ↓
10 ⇄ 20 ⇄ 30
```

We want:

```
head
 ↓
20 ⇄ 30
```

We move:

```
head = head.next;
```

Now:

```
head → 20 ⇄ 30
```

But we must also fix:

```
head.prev
```

The new first node should have:

```
prev = null
```

So:

```
head.prev = null;
```

18. Complete Delete-First

```
void deleteFirst() {

    if (head == null) {
        return;
    }

    if (head == tail) {
        head = tail = null;
        return;
    }

    head = head.next;

    head.prev = null;
}
```

Why:

```
head == tail
```

?

Because if there's only one node:

```
head
 ↓
10
 ↑
tail
```

After deletion:

```
head = null
tail = null
```

Both must be updated.

19. Delete Last

Suppose:

```
head             tail
 ↓                 ↓
10 ⇄ 20 ⇄ 30
```

Delete 30.

Because we have tail, we don't need traversal.

Move:

```
tail = tail.prev;
```

Now:

```
10 ⇄ 20
      ↑
     tail
```

But 20.next still points to the old 30.

So:

```
tail.next = null;
```

Result:

```
10 ⇄ 20 → null
```

20. Complete Delete-Last

```
void deleteLast() {

    if (tail == null) {
        return;
    }

    if (head == tail) {
        head = tail = null;
        return;
    }

    tail = tail.prev;

    tail.next = null;
}
```

This is one major advantage of maintaining tail.

21. Delete a Specific Node

Suppose:

```
10 ⇄ 20 ⇄ 30 ⇄ 40
```

We want to delete 30.

We have:

```
prev = 20
current = 30
next = 40
```

Before:

```
20 ⇄ 30 ⇄ 40
```

We want:

```
20 ⇄ 40
```

Therefore:

```
current.prev.next = current.next;
```

This changes:

```
20.next = 40
```

Then:

```
current.next.prev = current.prev;
```

This changes:

```
40.prev = 20
```

Now:

```
20 ⇄ 40
```

22. Complete Delete-Node Logic

```java
void deleteNode(Node current) {

    if (current == null) {
        return;
    }

    if (current == head) {
        deleteFirst();
        return;
    }

    if (current == tail) {
        deleteLast();
        return;
    }

    current.prev.next = current.next;

    current.next.prev = current.prev;
}
```

The two critical lines are:

```java
current.prev.next = current.next;
current.next.prev = current.prev;
```

Remember them as:

```
Previous → Next
Next → Previous
```

Both directions must be repaired.

23. Complete Doubly Linked List

Now let's put the fundamental operations together.

```java
class DoublyLinkedList {

    class Node {

        int data;
        Node prev;
        Node next;

        Node(int data) {
            this.data = data;
        }
```

```
    }

    Node head;
    Node tail;

    void insertFirst(int value) {

        Node newNode = new Node(value);

        if (head == null) {
            head = tail = newNode;
            return;
        }

        newNode.next = head;
        head.prev = newNode;

        head = newNode;
    }

    void insertLast(int value) {

        Node newNode = new Node(value);

        if (head == null) {
            head = tail = newNode;
            return;
        }

        tail.next = newNode;
        newNode.prev = tail;

        tail = newNode;
    }

    void deleteFirst() {

        if (head == null) {
            return;
        }

        if (head == tail) {
            head = tail = null;
            return;
        }

        head = head.next;
        head.prev = null;
    }

    void deleteLast() {

        if (tail == null) {
            return;
        }

        if (head == tail) {
            head = tail = null;
            return;
        }

        tail = tail.prev;
        tail.next = null;
```

```
    }

    void displayForward() {

        Node current = head;

        while (current != null) {

            System.out.print(current.data + " ");

            current = current.next;
        }

        System.out.println();
    }

    void displayBackward() {

        Node current = tail;

        while (current != null) {

            System.out.print(current.data + " ");

            current = current.prev;
        }

        System.out.println();
    }
}
```

24. Doubly Linked List Complexity

With both head and tail:

Operation	Time
Insert first	O(1)
Insert last	O(1)
Delete first	O(1)
Delete last	O(1)
Traverse forward	O(n)
Traverse backward	O(n)
Search	O(n)
Access by index	O(n)

The important advantage is:

```
head + tail
     ↓
operations at both ends are O(1)
```

Part 2 — Circular Linked List

Now let's understand the other major type.

25. What is a Circular Linked List?

Normal Singly Linked List:

```
10 → 20 → 30 → null
```

Circular Linked List:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

The biggest difference:

Normal

```
last.next = null;
```

Circular

```
last.next = head;
```

That's the fundamental idea.

26. Why is it called Circular?

Because there is no real ending.

Starting at 10:

```
10 → 20 → 30 → 10 → 20 → 30 → ...
```

You can keep going forever.

There is no:

```
null
```

at the end.

27. Node Structure

A Circular Singly Linked List uses the same Node structure:

```
class Node {

    int data;
    Node next;

    Node(int data) {
        this.data = data;
    }
}
```

The difference is how we connect the last node.

Normal:

```
30.next = null
```

Circular:

```
30.next = head
```

28. Creating a Circular List Manually

Create:

```
Node first = new Node(10);
Node second = new Node(20);
Node third = new Node(30);
```

Connect:

```
first.next = second;
second.next = third;
third.next = first;
```

Now:

```
       ┌──────────────┐
       ↓              │
10 → 20 → 30 ─────────┘
```

Or:

```
       10
      /  \
     ↑    ↓
    30 ← 20
```

The important connection is:

```
third.next = first;
```

29. Using head

As usual:

```
Node head = first;
```

Now:

```
head
 ↓
10 → 20 → 30
↑          ↓
└──────────┘
```

30. Why Normal Traversal Doesn't Work

For a normal Linked List:

```
while (current != null)
```

works because eventually:

```
current → null
```

But in a Circular Linked List:

```
10 → 20 → 30 → 10 → 20 → ...
```

current will never become null.

So:

```
while (current != null)
```

creates an infinite loop.

31. Correct Circular Traversal

We stop when we come back to head.

```
Node current = head;

do {

    System.out.println(current.data);

    current = current.next;

} while (current != head);
```

This is one of the most important differences.

32. Dry Run of Circular Traversal

Suppose:

```
10 → 20 → 30 → 10
```

Initially:

```
current = head
```

Therefore:

```
current → 10
```

Print:

```
10
```

Move:

```
current = current.next;
```

Now:

```
current → 20
```

Print:

```
20
```

Move:

```
current → 30
```

Print:

```
30
```

Move:

```
current → 10
```

Now:

```
current == head
```

So stop.

Output:

```
10
20
30
```

33. Why do-while is Useful

Consider an empty circular list:

```
head = null
```

Then we must handle it before:

```
current.data
```

So:

```
if (head == null) {
    return;
}
```

Then:

```
Node current = head;

do {
    ...
} while (current != head);
```

The do-while structure naturally processes the first node once before checking whether we've returned to the beginning.

34. Insert at Beginning in Circular Linked List

Suppose:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

We want:

```
5 → 10 → 20 → 30
↑               ↓
└───────────────┘
```

Create:

```
Node newNode = new Node(5);
```

We need:

```
5 → 10
```

So:

```
newNode.next = head;
```

But there's another important connection.

The last node currently points to:

```
head = 10
```

After inserting 5, the last node should point to:

```
5
```

So we need:

```
last.next = newNode;
```

Then:

```
head = newNode;
```

35. Insert First — Circular Code

Without a tail pointer:

```
void insertFirst(int value) {

    Node newNode = new Node(value);

    if (head == null) {

        head = newNode;

        newNode.next = head;

        return;
    }

    Node current = head;

    while (current.next != head) {
        current = current.next;
    }

    newNode.next = head;

    current.next = newNode;

    head = newNode;
}
```

The important condition:

```
current.next != head
```

means:

Keep moving until current is the last node.

Because the last node satisfies:

```
current.next == head
```

36. Circular List With a tail

There is a better way.

Instead of only:

```
Node head;
```

we can maintain:

```
Node head;
Node tail;
```

And maintain this rule:

```
tail.next == head
```

For:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

we have:

```
head → 10
tail → 30
```

and:

```
tail.next == head
```

37. Why tail Makes Circular Lists Easier

Suppose:

```
head
 ↓
10 → 20 → 30
          ↑
         tail
```

We want to insert 5 at beginning.

We don't need to search for 30.

We already know:

```
tail → 30
```

So:

```
newNode.next = head;
head = newNode;
tail.next = head;
```

Done.

38. Insert First With Head + Tail

```
void insertFirst(int value) {

    Node newNode = new Node(value);

    if (head == null) {

        head = tail = newNode;

        newNode.next = head;

        return;
    }

    newNode.next = head;

    head = newNode;

    tail.next = head;
}
```

Let's dry run.

Before:

```
head
 ↓
10 → 20 → 30
          ↑
         tail
```

New node:

```
5
```

After:

```
newNode.next = head;
```

```
5 → 10 → 20 → 30
```

Then:

```
head = newNode;
```

Now:

```
head
 ↓
5 → 10 → 20 → 30
```

Finally:

```
tail.next = head;
```

Now:

```
        ┌──────────────────┐
        ↓                  │
5 → 10 → 20 → 30 ──────────┘
↑                    ↑
head                tail
```

39. Insert at End With Tail

This becomes very easy.

Suppose:

```
head
 ↓
10 → 20 → 30
           ↑
          tail
```

Insert 40.

Create:

```
Node newNode = new Node(40);
```

We want:

```
30 → 40 → head
```

So:

```
newNode.next = head;
```

Then:

```
tail.next = newNode;
```

Then:

```
tail = newNode;
```

40. Complete Insert-Last

```
void insertLast(int value) {

    Node newNode = new Node(value);

    if (head == null) {

        head = tail = newNode;

        newNode.next = head;

        return;
    }

    newNode.next = head;

    tail.next = newNode;

    tail = newNode;
}
```

This is O(1) because we already have tail.

41. Delete First

Suppose:

```
head
 ↓
10 → 20 → 30
          ↑
         tail
```

Delete 10.

New head should be:

```
20
```

So:

```
head = head.next;
```

But remember:

```
tail.next
```

must point to the new head.

So:

```
tail.next = head;
```

Result:

```
head
 ↓
20 → 30
     ↑
    tail
```

and:

```
tail.next → head
```

42. Special Case: One Node

Suppose:

```
head
 ↓
10
 ↑
tail
```

and:

```
10.next = 10
```

If we delete it, the list becomes:

```
head = null
tail = null
```

So we need:

```
if (head == tail) {
    head = tail = null;
    return;
}
```

This is an important circular-list edge case.

43. Complete Delete-First

```
void deleteFirst() {

    if (head == null) {
        return;
    }

    if (head == tail) {
        head = tail = null;
        return;
    }

    head = head.next;

    tail.next = head;
}
```

44. Delete Last

Suppose:

```
10 → 20 → 30
↑          ↓
└──────────┘
```

We want:

```
10 → 20
↑      ↓
└──────┘
```

We need the node before tail.

That is:

```
20
```

We traverse until:

```
current.next == tail
```

Then:

```
current.next = head;
tail = current;
```

45. Complete Delete-Last

```
void deleteLast() {

    if (head == null) {
        return;
    }

    if (head == tail) {
        head = tail = null;
        return;
    }

    Node current = head;

    while (current.next != tail) {
        current = current.next;
    }

    current.next = head;

    tail = current;
}
```

Because we're using a singly circular list, deleting the last node is O(n).

46. Complete Circular Singly Linked List

```
class CircularLinkedList {

    class Node {

        int data;
        Node next;

        Node(int data) {
            this.data = data;
        }
    }

    Node head;
    Node tail;

    void insertFirst(int value) {

        Node newNode = new Node(value);

        if (head == null) {

            head = tail = newNode;

            newNode.next = head;

            return;
        }

        newNode.next = head;

        head = newNode;

        tail.next = head;
    }

    void insertLast(int value) {

        Node newNode = new Node(value);

        if (head == null) {

            head = tail = newNode;

            newNode.next = head;

            return;
        }

        newNode.next = head;

        tail.next = newNode;

        tail = newNode;
    }

    void deleteFirst() {

        if (head == null) {
            return;
        }
```

```
        if (head == tail) {
            head = tail = null;
            return;
        }

        head = head.next;

        tail.next = head;
    }

    void deleteLast() {

        if (head == null) {
            return;
        }

        if (head == tail) {
            head = tail = null;
            return;
        }

        Node current = head;

        while (current.next != tail) {
            current = current.next;
        }

        current.next = head;

        tail = current;
    }

    void display() {

        if (head == null) {
            return;
        }

        Node current = head;

        do {

            System.out.print(current.data + " ");

            current = current.next;

        } while (current != head);

        System.out.println();
    }
}
```

47. Circular Doubly Linked List

There's one more combination you should know.

We can combine:

```
Doubly Linked List
+
Circular Linked List
```

Each node has:

```
prev
data
next
```

But instead of:

```
head.prev = null
tail.next = null
```

we connect them circularly.

So:

```
head.prev = tail;
tail.next = head;
```

Example:



More precisely:

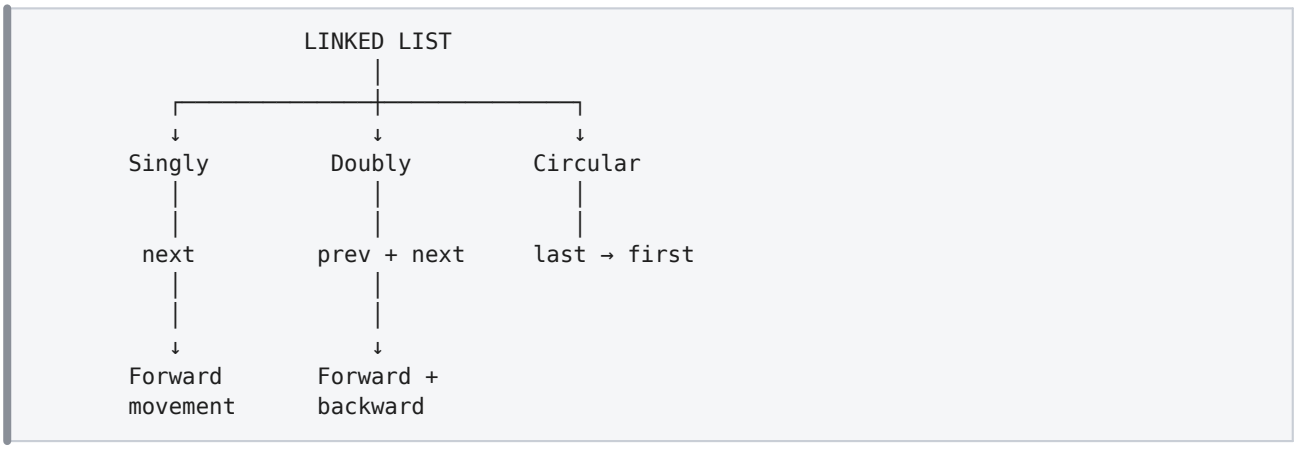
```
head = 10
tail = 40

10.prev = 40
40.next = 10
```

This allows movement in both directions forever.

48. Singly vs Doubly vs Circular

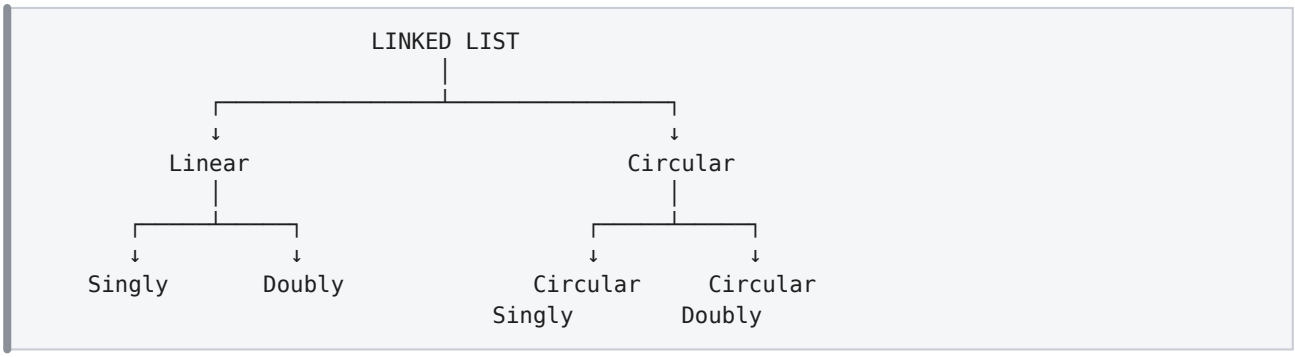
Now the Big Picture can properly be expanded.



But Circular can itself be:



So a better structure is:



49. The Key Differences

Feature	Singly	Doubly	Circular Singly
data	Yes	Yes	Yes
next	Yes	Yes	Yes
prev	No	Yes	No
Last → null	Yes	Yes	No
Last → first	No	No	Yes
Forward traversal	Yes	Yes	Yes
Backward traversal	No	Yes	No
Extra memory	Low	Higher	Low

50. When Should You Use Each?

Singly Linked List

Good when you primarily need:

```
forward traversal
+
simple node structure
```

Common DSA problems use this form.

Doubly Linked List

Useful when you need:

```
forward + backward movement
```

For example:

```
previous page ↔ current page ↔ next page
```

It is also useful in structures such as an LRU cache, where constant-time removal and movement of nodes are important when combined with a hash map.

Circular Linked List

Useful when the process should repeatedly cycle:

```
A → B → C → A → B → C → ...
```

Examples include:

```
Round-robin scheduling
Circular processing
Repeated turns
```

51. The Most Important Doubly Linked List Pattern

When changing a middle node, remember:

```
A ⇄ B ⇄ C
```

If deleting B:

```
A ⇄ C
```

you need two changes:

```
B.prev.next = B.next;
B.next.prev = B.prev;
```

Think:

```
Previous → Next
Next → Previous
```

52. The Most Important Circular Linked List Rule

If using head and tail, remember:

```
tail.next = head;
```

Always.

For example:

```
head
 ↓
10 → 20 → 30
          ↑
         tail
```

The invariant is:

```
tail.next == head
```

After every insertion/deletion, make sure that remains true.

That one rule makes Circular Linked Lists much easier to reason about.